

Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Інститут комп’ютерних наук та інформаційних технологій



Лабораторно робота №5
з дисципліни: “Технології доповненої реальності”
на тему:
“UI та жести в AR”

Виконав:
ст. групи ПП-44
ВЕРЕЩАК Богдан

Прийняв:
ас. Юрій ПАТЕРЕГА

Львів - 2026

Мета роботи

Після виконання лабораторної роботи студент зможе:

- створювати UI-елементи в AR-додатку (Canvas, кнопки, текст);
- реалізувати вибір об'єкта через Raycast (Physics.Raycast для 3D-об'єктів);
- реалізувати жест Pinch-to-Scale (масштабування двома пальцями);
- реалізувати обертання об'єкта одним пальцем;
- поєднувати UI-елементи з AR-взаємодією.

Індивідуальне завдання

$$N = 3, P = 7$$

$$V = ((N - 1 + (P - 1) \times 2) \% 15) + 1 = (14 \% 15) + 1 = 15$$

$$M = ((N - 1 + P) \% 4) \rightarrow 0 = A, 1 = B, 2 = C, 3 = D, \quad M = 1 = B$$

М	К-сть об'єктів	Позиція UI	Деталі Layout
В	4	Зверху	Horizontal Layout, anchor top-center

V	Завдання	Підказка	Перевірка виконання
15	Реалізувати змінну форми вибраного об'єкта (куб → сфера → циліндр)	Змінити MeshFilter.mesh або видалити/створити новий	Кнопка змінює форму; Collider відповідає; колір зберігається

Хід роботи

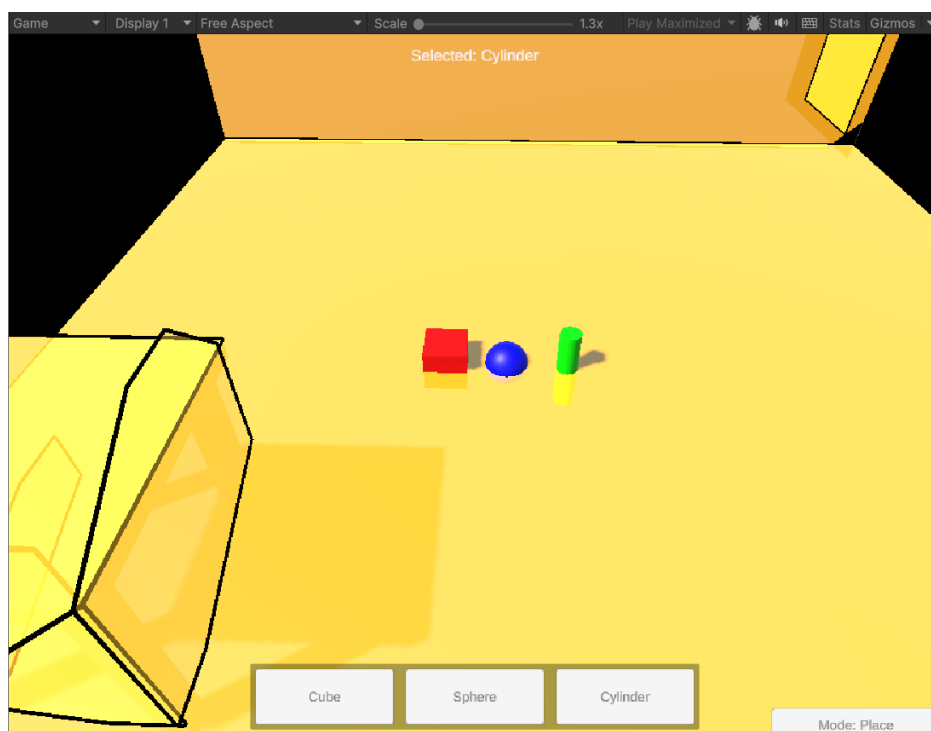


Рис. 1. Розміщення об'єкта загальне завдання

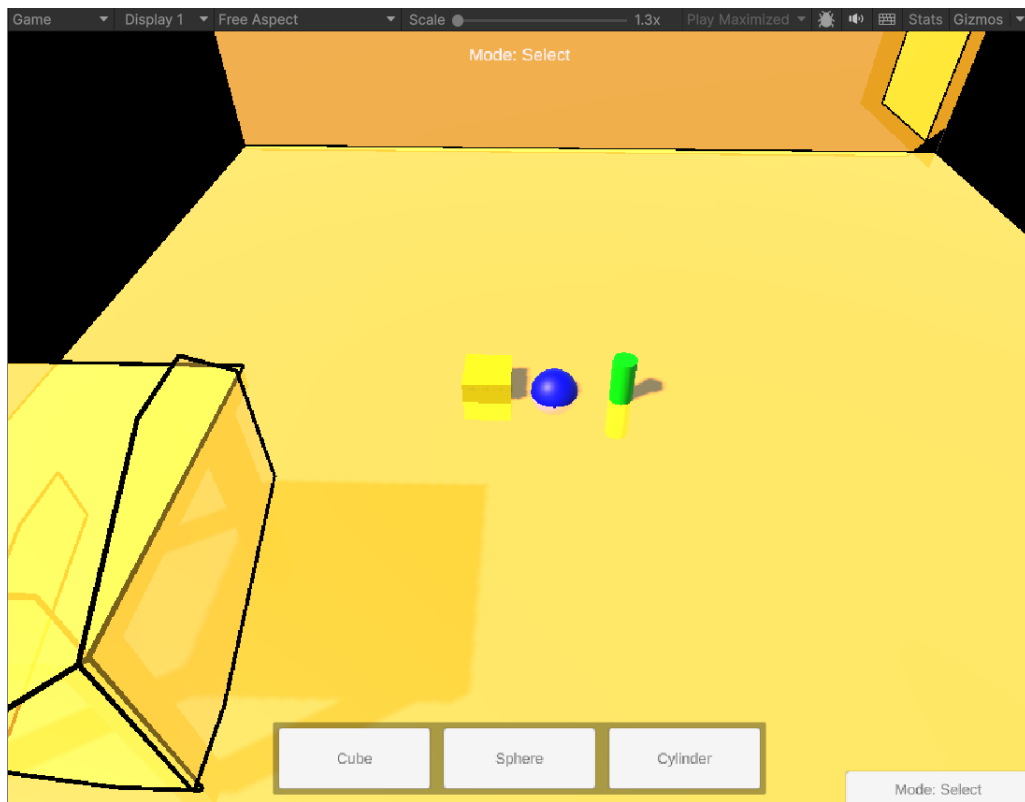


Рис. 2. Виділення об'єкта загальне завдання

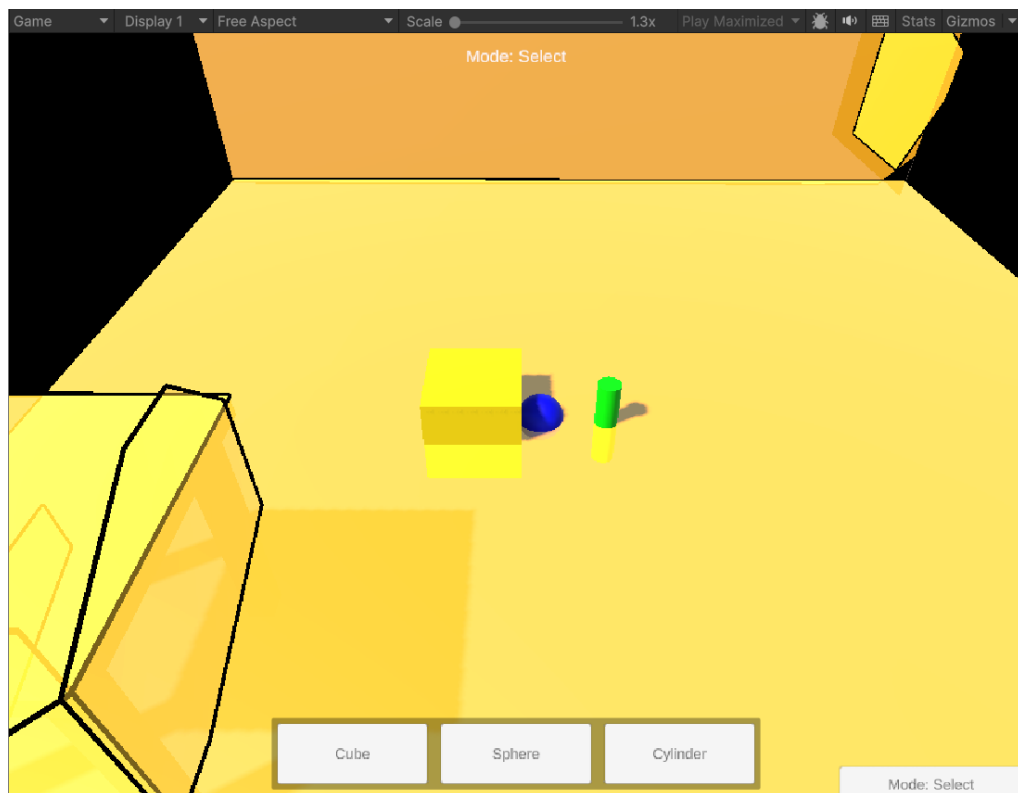


Рис. 3. Масштабування об'єкта загальне завдання

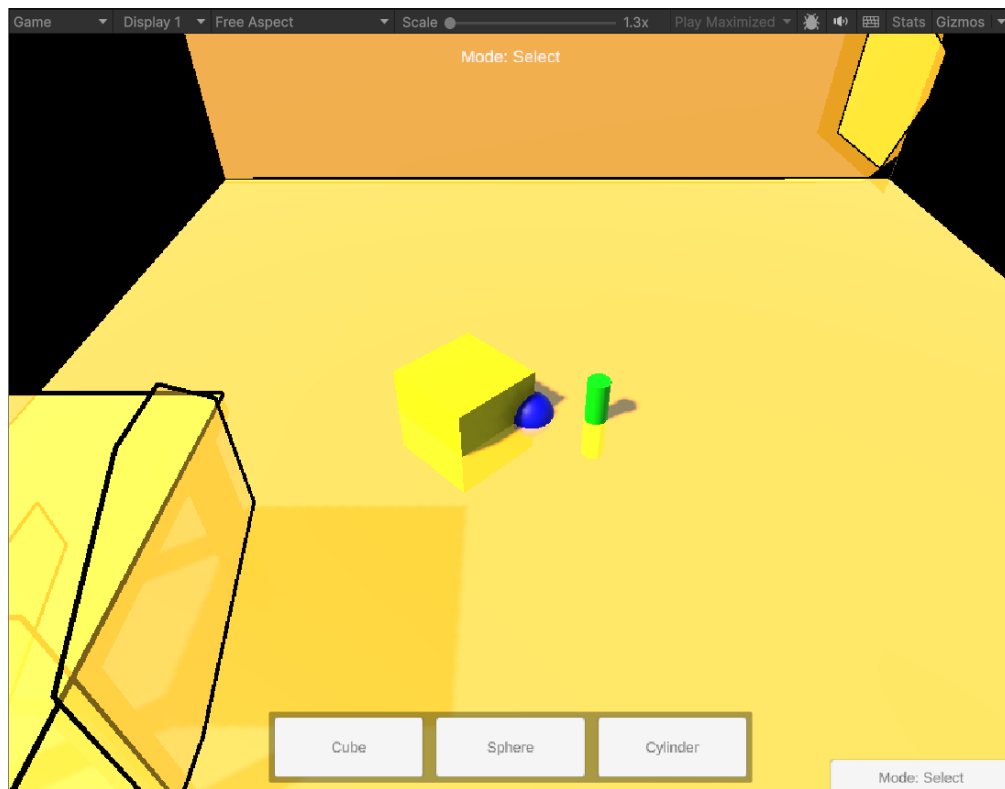


Рис. 4. Обертання об'єкта загальне завдання

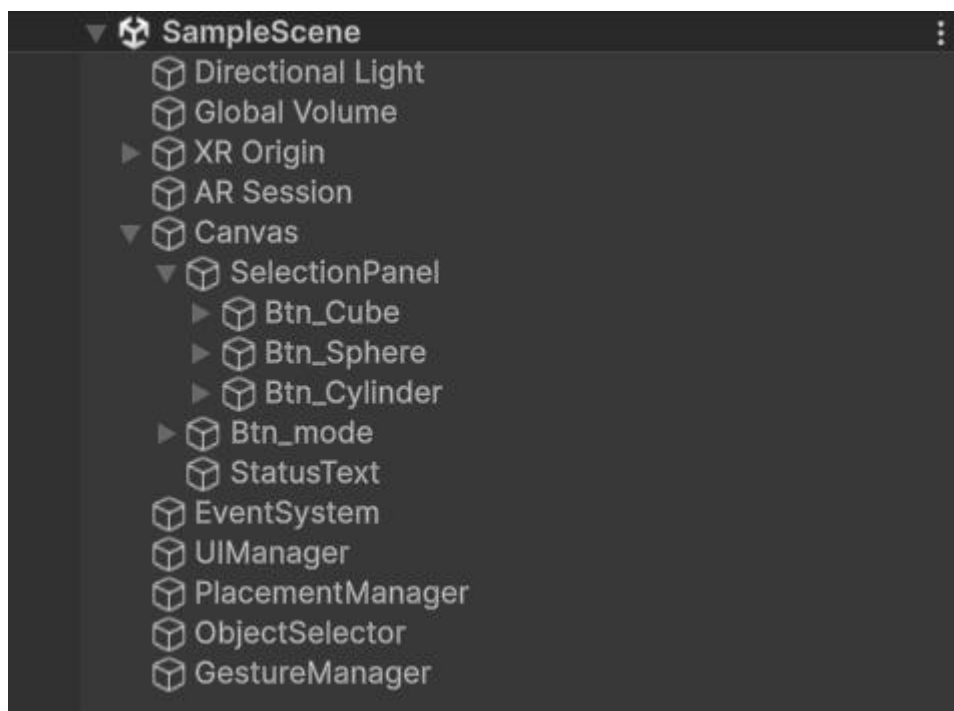


Рис. 5. Ієрархія фінальної сцени

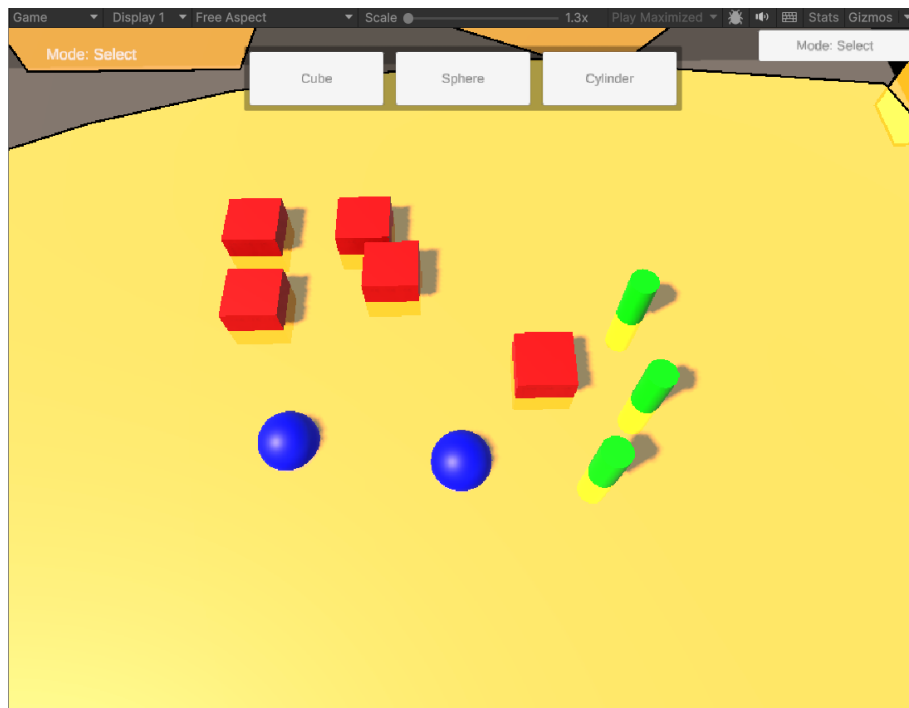


Рис. 6. Розміщення UI зверху індивідуальне завдання

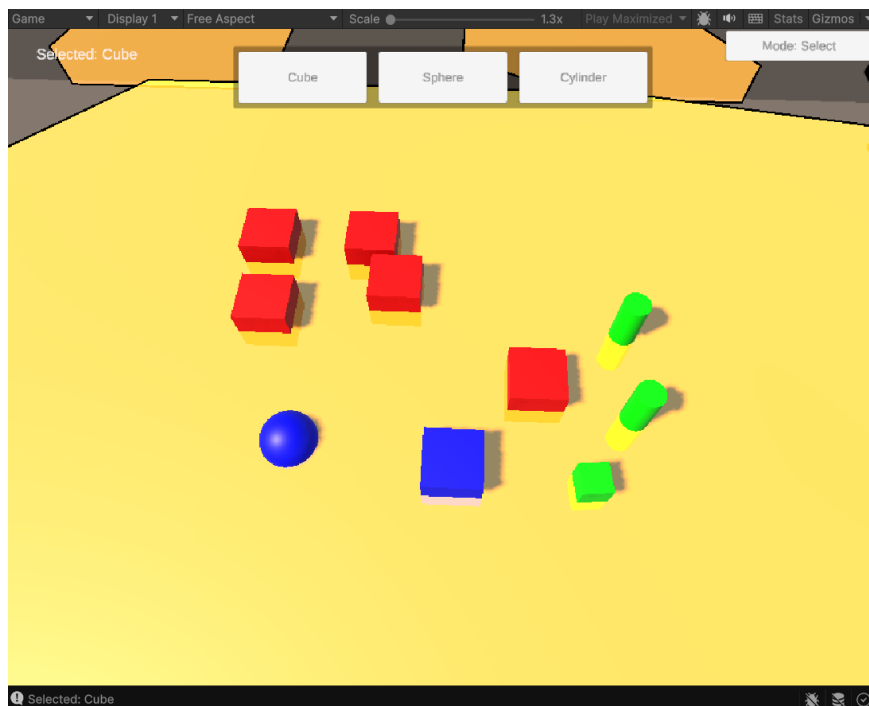


Рис. 7. Зміна обраного об'єкта → куб індивідуальне завдання



Рис. 8. Зміна обраного об'єкта → сфера індивідуальне завдання

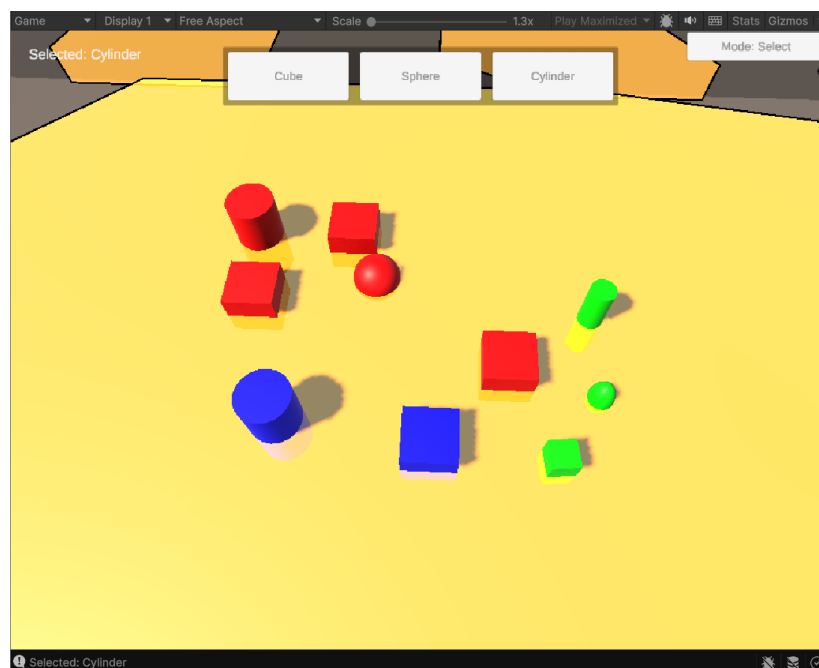


Рис. 9. Зміна обраного об'єкта → циліндр індивідуальне завдання

Код індивідуального завдання (зміни від початкового коду вказані жирним курсивом)

ObjectSelector.cs:

```
using UnityEngine;
using UnityEngine.EventSystems;
public class ObjectSelector : MonoBehaviour
{
    [Header("References")]
    [SerializeField] private UIManager uiManager;
    [Header("Highlight")]
```

```

[SerializeField] private Color highlightColor = Color.yellow;
private GameObject selectedObject;
private Color originalColor;
private Renderer selectedRenderer;
void Update()
{
    if (uiManager.GetCurrentMode() != UIManager.InteractionMode.Select)
        return;
    if (Input.touchCount == 1)
    {
        Touch touch = Input.GetTouch(0);
        if (touch.phase != TouchPhase.Began) return;
        if (EventSystem.current.IsPointerOverGameObject(touch.fingerId)) return;
        TrySelectObject(touch.position);
    }
    else if (Input.GetMouseButtonDown(0))
    {
        if (EventSystem.current.IsPointerOverGameObject()) return;
        TrySelectObject(Input.mousePosition);
    }
}
public void ChangeSelectedMesh(Mesh newMesh, Material newMaterial)
{
    if (selectedObject == null) return;
    selectedObject.GetComponent<MeshFilter>().mesh = newMesh;
    MeshCollider collider = selectedObject.GetComponent<MeshCollider>();
    if (collider != null) collider.sharedMesh = newMesh;
}
private void TrySelectObject(Vector2 screenPosition)
{
    Ray ray = Camera.main.ScreenPointToRay(screenPosition);
    RaycastHit hit;
    if (Physics.Raycast(ray, out hit, 100f))
    {
        GameObject hitObject = hit.collider.gameObject;
        DeselectCurrent();
        selectedObject = hitObject;
        selectedRenderer = hitObject.GetComponent<Renderer>();
        if (selectedRenderer != null)
        {
            originalColor = selectedRenderer.material.color;
            selectedRenderer.material.color = highlightColor;
        }
        Debug.Log($"Selected: {hitObject.name}");
    }
    else { DeselectCurrent(); }
}
private void DeselectCurrent()
{
    if (selectedRenderer != null)
        selectedRenderer.material.color = originalColor;
    selectedObject = null;
    selectedRenderer = null;
}

```

```

    public GameObject GetSelectedObject() { return selectedObject; }
}

```

UIManager.cs:

```

using UnityEngine;
using TMPro;
public class UIManager : MonoBehaviour
{
    public enum InteractionMode { Place, Select }
    [Header("Prefabs")]
    [SerializeField] private GameObject cubePrefab;
    [SerializeField] private GameObject spherePrefab;
    [SerializeField] private GameObject cylinderPrefab;
    [Header("UI")]
    [SerializeField] private TextMeshProUGUI statusText;
    [SerializeField] private TextMeshProUGUI modeButtonText;
    [SerializeField] private ObjectSelector selector;
    private InteractionMode currentMode = InteractionMode.Place;
    private GameObject selectedPrefab;
    private Mesh selectedMesh;
    private Material selectedMaterial;
    void Start()
    {
        SelectCube();
        UpdateModeUI();
    }
    public void SelectCube()
    {
        SetSelection(cubePrefab);
        UpdateStatus("Selected: Cube");
    }
    public void SelectSphere()
    {
        SetSelection(spherePrefab);
        UpdateStatus("Selected: Sphere");
    }
    public void SelectCylinder()
    {
        SetSelection(cylinderPrefab);
        UpdateStatus("Selected: Cylinder");
    }
    private void SetSelection(GameObject prefab)
    {
        selectedPrefab = prefab;
        selectedMesh = prefab.GetComponent<MeshFilter>().sharedMesh;
        selectedMaterial = prefab.GetComponent<MeshRenderer>().sharedMaterial;
        if (currentMode == InteractionMode.Select)
        {
            GameObject active = selector.GetSelectedObject();
            if (active != null)
            {
                Mesh newMesh = prefab.GetComponent<MeshFilter>().sharedMesh;
                Material newMat = prefab.GetComponent<MeshRenderer>().sharedMaterial;
                selector.ChangeSelectedMesh(newMesh, newMat);
            }
        }
    }
}

```



```

    }
}
public void ToggleMode()
{
    currentMode = (currentMode == InteractionMode.Place)
        ? InteractionMode.Select
        : InteractionMode.Place;
    UpdateModeUI();
}
public InteractionMode GetCurrentMode() => currentMode;
public GameObject GetSelectedPrefab() => selectedPrefab;
public Mesh GetSelectedMesh() => selectedMesh;
public Material GetSelectedMaterial() => selectedMaterial;
private void UpdateModeUI()
{
    string modeText = currentMode == InteractionMode.Place
        ? "Mode: Place"
        : "Mode: Select";
    if (modeButtonText != null)
        modeButtonText.text = modeText;
    UpdateStatus(modeText);
}
private void UpdateStatus(string message)
{
    if (statusText != null)
        statusText.text = message;
    Debug.Log(message);
}
}

```

Відео демонстрація екрану телефону розробленого AR додатку

https://drive.google.com/file/d/1H-DWRZKj_1bLmsKypqaFYwt6UEBr_Hgm/view?usp=sharing

Висновок

На даній лабораторній роботі я навчився створювати UI-елементи в AR-додатку (Canvas, кнопки, текст), реалізувати вибір об'єкта через Raycast (Physics.Raycast для 3D-об'єктів), реалізувати жест Pinch-to-Scale (масштабування двома пальцями), реалізувати обертання об'єкта одним пальцем, поєднувати UI-елементи з AR-взаємодією.